🧩 Module – Concurrency Overview and Thread Basics

What Is Concurrency?

Key Points
• **Concurrency** means executing multiple tasks that may overlap in time.
• **Parallelism** executes tasks simultaneously on multiple cores; concurrency is broader — managing multiple tasks efficiently.
• Java supports concurrency through **threads**, **executors**, and **synchronization constructs**.
• Concurrency improves responsiveness, resource utilization, and performance for I/O-bound or CPU-bound operations.

Code Snippet

```
System.out.println("Main starts");
new Thread(() -> System.out.println("Worker running")).start();
System.out.println("Main ends");
```

Exercise 01 – ThreadIntro.java
Create a program that prints messages from the main thread and a worker thread simultaneously.

The Thread Class

Key Points
• Every Java thread is an instance of `java.lang.Thread`.
• You can create threads by:
– Extending `Thread` and overriding `run()`.
– Passing a `Runnable` or `Callable` object to a `Thread` constructor.
• Start execution using the `start()` method (not `run()` directly).

Code Snippet

```
class Worker extends Thread {
    public void run() {
        System.out.println("Running in " +
Thread.currentThread().getName());
    }
}
new Worker().start();
```

Exercise 02 – ThreadRun.java
Create two threads using both the subclass and `Runnable` approaches.

Thread Lifecycle

Key Points
• A thread moves through several states:
NEW → RUNNABLE → BLOCKED/WAITING → TERMINATED.
• The `start()` method transitions a thread from NEW to RUNNABLE.
• The JVM scheduler decides when threads actually run.
• Use `join()` to wait for another thread to finish.

Code Snippet

```
Thread t = new Thread(() -> System.out.println("Running"));
t.start();
t.join();
System.out.println("Done");
```

Exercise 03 – ThreadStates.java
Print messages from within different thread lifecycle stages to observe transitions.

Thread Priorities and Naming

Key Points
• Threads have priorities (1–10) influencing scheduling but not guaranteeing order.
• You can set and get thread names and priorities.
• Use naming to identify threads during debugging.

Code Snippet

```
Thread t = new Thread(() -> System.out.println("Thread executed"));
t.setName("Learnato-Worker");
t.setPriority(Thread.NORM_PRIORITY + 1);
t.start();
```

Exercise 04 – ThreadNaming.java
Create three threads with custom names and different priorities. Observe execution order.

Runnable vs Callable

Key Points
• `Runnable` represents a task with no return value.
• `Callable<V>` represents a task returning a result and may throw checked exceptions.
• Used with `ExecutorService` to manage thread pools efficiently.

Code Snippet

```
Callable<Integer> task = () -> 42;
ExecutorService ex = Executors.newSingleThreadExecutor();
Future<Integer> result = ex.submit(task);
System.out.println(result.get());
ex.shutdown();
```

Exercise 05 – CallableExample.java
Create a `Callable` that calculates the factorial of a number and prints the result.

Synchronization and Shared Resources

Key Points
• Threads sharing mutable data can cause race conditions.
• Use `synchronized` blocks or locks to ensure mutual exclusion.
• Synchronization ensures only one thread accesses a shared resource at a time.

Code Snippet

```
class Counter {
    private int count = 0;
    synchronized void increment() { count++; }
    int get() { return count; }
}
```

Exercise 06 – SyncCounter.java
Create a synchronized counter and update it from multiple threads to verify correct results.

Thread Interruption and Daemon Threads

Key Points
• Use `interrupt()` to signal a thread to stop politely.
• Threads should check `isInterrupted()` periodically.
• Daemon threads run in the background and terminate automatically when all user threads finish.

Code Snippet

```
Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        // work
    }
});
worker.start();
worker.interrupt();
```

Exercise 07 – ThreadInterrupt.java
Implement a long-running loop that gracefully stops when interrupted.

Thread Safety and Atomic Operations

Key Points
• Thread safety ensures that multiple threads access shared data without corruption.
• Use `java.util.concurrent.atomic` classes for lightweight atomic updates.
• Common examples: `AtomicInteger`, `AtomicBoolean`, `AtomicReference`.

Code Snippet

```
AtomicInteger counter = new AtomicInteger(0);
IntStream.range(0, 1000)
         .parallel()
         .forEach(i -> counter.incrementAndGet());
System.out.println(counter.get());
```

Exercise 08 – AtomicCounter.java
Use `AtomicInteger` to count increments safely across multiple threads.

Executors and Thread Pools

Key Points

• The `ExecutorService` framework simplifies thread management.

• Types of executors:

– `newFixedThreadPool(n)`

– `newCachedThreadPool()`

– `newSingleThreadExecutor()`

– `newScheduledThreadPool()`

• Always call `shutdown()` or `shutdownNow()` after use.

Code Snippet

```
ExecutorService pool = Executors.newFixedThreadPool(3);
for (int i = 0; i < 5; i++) {
    pool.submit(() ->
System.out.println(Thread.currentThread().getName()));
}
pool.shutdown();
```

Exercise 09 – ThreadPoolDemo.java

Submit multiple tasks to a fixed thread pool and observe parallel execution.

Certification Tips

Key Points
• Threads created via `new Thread()` or `Executors`.
• `start()` starts a thread; calling `run()` directly runs in the current thread.
• Synchronization ensures thread safety but may reduce performance.
• `Callable` returns results, unlike `Runnable`.
• Expect exam questions about thread states, synchronization keywords, and executor usage.

Code Snippet

```
Future<String> f = Executors.newSingleThreadExecutor()
    .submit(() -> "Task done");
System.out.println(f.get());
```

Exercise 10 – CertificationPractice.java
Identify the output and state transitions for a multithreaded program using `Thread`, `Runnable`, and `ExecutorService`.

🧩 Module – Advanced Concurrency (Locks, Synchronizers, Concurrent Collections, CompletableFuture)

Explicit Locks and ReentrantLock

Key Points
• `ReentrantLock` offers more control than the `synchronized` keyword.
• Supports features like timed lock attempts and fair locking.
• Must always release the lock in a `finally` block to avoid deadlocks.

Code Snippet

```
Lock lock = new ReentrantLock();
try {
    lock.lock();
    System.out.println("Critical section by " +
Thread.currentThread().getName());
} finally {
    lock.unlock();
}
```

Exercise 01 – LockDemo.java
Use `ReentrantLock` to protect a shared counter. Start multiple threads incrementing it safely.

TryLock and Fair Locking

Key Points
• `tryLock()` lets you attempt acquiring a lock without waiting indefinitely.
• Returns `true` if lock acquired, `false` otherwise.
• Use the fair mode (`new ReentrantLock(true)`) to serve threads in FIFO order.

Code Snippet

```
Lock lock = new ReentrantLock(true);
if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {
    try { System.out.println("Lock acquired"); }
    finally { lock.unlock(); }
} else {
    System.out.println("Lock unavailable");
}
```

Exercise 02 – TryLockExample.java
Simulate multiple threads attempting to acquire a shared lock with a timeout using `tryLock()`.

ReadWriteLock

Key Points
• `ReadWriteLock` allows multiple readers but only one writer.
• Useful when reads are frequent and writes are rare.
• Implemented by `ReentrantReadWriteLock`.

Code Snippet

```
ReadWriteLock rw = new ReentrantReadWriteLock();
rw.readLock().lock();
try { System.out.println("Reading..."); }
finally { rw.readLock().unlock(); }
```

Exercise 03 – ReadWriteExample.java
Implement a cache using `ReadWriteLock` where multiple threads can read but only one can update.

Synchronizers Overview

Key Points
• Synchronizers coordinate thread execution and manage resource sharing.
• Common synchronizers:
– `CountDownLatch` → waits for a set of tasks to complete.
– `CyclicBarrier` → allows threads to meet at a synchronization point.
– `Semaphore` → controls number of concurrent accesses.
– `Exchanger` → allows threads to exchange data.

Code Snippet

```
CountDownLatch latch = new CountDownLatch(3);
for (int i = 0; i < 3; i++) {
    new Thread(() -> { System.out.println("Worker done");
latch.countDown(); }).start();
}
latch.await();
System.out.println("All workers finished");
```

Exercise 04 – LatchBarrierDemo.java
Demonstrate the difference between `CountDownLatch` and `CyclicBarrier` in a simulation of multiple worker threads.

Semaphore and Resource Control

Key Points
• `Semaphore` limits concurrent access to a resource.
• Threads acquire a permit before proceeding and release it when done.
• Commonly used to throttle concurrent operations.

Code Snippet

```
Semaphore sem = new Semaphore(2);
for (int i = 0; i < 5; i++) {
    new Thread(() -> {
        try {
            sem.acquire();
            System.out.println(Thread.currentThread().getName() + "
acquired permit");
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
        finally { sem.release(); }
    }).start();
}
```

Exercise 05 – SemaphoreExample.java
Simulate a download manager where only two files can be downloaded concurrently using `Semaphore`.

Concurrent Collections

Key Points
• Java provides thread-safe collections optimized for concurrency.
• Key interfaces: `ConcurrentMap`, `BlockingQueue`, `ConcurrentNavigableMap`.
• Common implementations:
– `ConcurrentHashMap`
– `CopyOnWriteArrayList`
– `LinkedBlockingQueue`
• Avoid explicit synchronization for simple concurrent data structures.

Code Snippet

```
ConcurrentMap<String, Integer> map = new ConcurrentHashMap<>();
IntStream.range(0, 10).parallel().forEach(i -> map.put("key" + i, i));
System.out.println(map.size());
```

Exercise 06 – ConcurrentMapDemo.java
Use `ConcurrentHashMap` to count word frequencies from multiple threads safely.

Blocking Queues

Key Points
• Blocking queues automatically manage producer-consumer coordination.
• Producers block when the queue is full; consumers block when it's empty.
• Implementations include `ArrayBlockingQueue`, `LinkedBlockingQueue`, and `PriorityBlockingQueue`.

Code Snippet

```
BlockingQueue<String> queue = new LinkedBlockingQueue<>(2);
new Thread(() -> {
    try { queue.put("A"); queue.put("B"); queue.put("C"); }
    catch (InterruptedException e) {}
}).start();
new Thread(() -> {
    try { System.out.println(queue.take()); }
    catch (InterruptedException e) {}
}).start();
```

Exercise 07 – ProducerConsumer.java
Implement a producer-consumer example using `LinkedBlockingQueue`.

CompletableFuture Overview

Key Points
• `CompletableFuture` provides a high-level API for asynchronous programming.
• Enables chaining of tasks and handling results or exceptions non-blockingly.
• Supports composition (`thenApply`, `thenCombine`, `thenAccept`).

Code Snippet

```
CompletableFuture.supplyAsync(() -> "Learnato")
    .thenApply(String::toUpperCase)
    .thenAccept(System.out::println);
```

Exercise 08 – FutureChain.java
Create a `CompletableFuture` chain that loads data, transforms it, and prints it asynchronously.

Combining and Handling Futures

Key Points
• Combine multiple asynchronous tasks using `thenCombine()` and `allOf()`.
• Handle errors gracefully with `exceptionally()` and `handle()`.
• Avoid blocking operations like `get()` unless absolutely necessary.

Code Snippet

```
CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> 20);
f1.thenCombine(f2, Integer::sum)
  .thenAccept(System.out::println);
```

Exercise 09 – CombineFutures.java
Run two asynchronous computations and combine their results using `thenCombine()`.

Certification Tips

Key Points

- `ReentrantLock` offers fine-grained locking and must be released manually.
- `CountDownLatch` vs `CyclicBarrier`: one-time vs reusable barrier.
- `Semaphore` limits concurrent resource access.
- `ConcurrentHashMap` avoids `ConcurrentModificationException`.
- `CompletableFuture` simplifies async chaining and exception handling.
- Expect exam questions involving thread coordination and async APIs.

Code Snippet

```
CompletableFuture.supplyAsync(() -> 5)
    .thenApply(x -> x * 2)
    .thenAccept(System.out::println);
```

Exercise 10 – CertificationPractice.java

Identify which concurrency utility (`Lock`, `Semaphore`, `Barrier`, or `Future`) fits each scenario in a multi-threaded application.

🧩 Module – Java I/O and NIO (Files, Streams, Channels, and Buffers)

Introduction to Java I/O

Key Points
• Java I/O provides APIs to read and write data from files, networks, and memory.
• Two main I/O systems:
– **Classic I/O (java.io)** → Stream-based, blocking.
– **NIO (java.nio)** → Buffer-based, non-blocking, faster for large data.
• Streams represent a continuous flow of data — either input or output.

Code Snippet

```
try (FileInputStream fis = new FileInputStream("data.txt")) {
    int b;
    while ((b = fis.read()) != -1) System.out.print((char) b);
}
```

Exercise 01 – FileReadBasic.java
Create a simple file reader using `FileInputStream` to print a text file's contents.

Byte Streams vs Character Streams

Key Points
• **Byte Streams** → for binary data (e.g., images, PDFs). Classes: `FileInputStream`, `FileOutputStream`.
• **Character Streams** → for text data, automatically handle encoding. Classes: `FileReader`, `FileWriter`.
• Always close streams (prefer try-with-resources).

Code Snippet

```
try (FileReader reader = new FileReader("input.txt");
     FileWriter writer = new FileWriter("copy.txt")) {
    int ch;
    while ((ch = reader.read()) != -1) writer.write(ch);
}
```

Exercise 02 – CopyFile.java
Copy a text file using `FileReader` and `FileWriter`.

Buffered Streams

Key Points
• Buffered streams improve efficiency by reducing disk I/O operations.
• Use `BufferedReader` and `BufferedWriter` for text, `BufferedInputStream` for binary data.
• Always wrap base streams inside buffered ones.

Code Snippet

```
try (BufferedReader br = new BufferedReader(new FileReader("notes.txt"))) {
    br.lines().forEach(System.out::println);
}
```

Exercise 03 – BufferedReaderDemo.java
Read a large text file using `BufferedReader` and print only lines containing a keyword.

Object Streams and Serialization

Key Points
• Serialization converts Java objects into byte streams for storage or transmission.
• Implement `Serializable` interface to mark classes as serializable.
• Use `ObjectOutputStream` and `ObjectInputStream` for writing/reading objects.

Code Snippet

```
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("user.dat"))) {
    oos.writeObject(new User("Rohan", 35));
}
```

Exercise 04 – ObjectSaveLoad.java
Create a `User` class implementing `Serializable`, write and read it using object streams.

Transient and serialVersionUID

Key Points
• `transient` fields are skipped during serialization.
• `serialVersionUID` ensures class compatibility during deserialization.
• Always define it explicitly for version control.

Code Snippet

```
class User implements Serializable {
    private static final long serialVersionUID = 1L;
    transient String password;
    String username;
}
```

Exercise 05 – TransientField.java
Demonstrate the effect of `transient` by serializing and deserializing an object with sensitive data.

Introduction to NIO

Key Points
• NIO (New I/O) introduced in Java 1.4 for faster, non-blocking operations.
• Uses **Buffers** for data storage and **Channels** for data transfer.
• Supports file operations, sockets, and memory-mapped files.
• Located in `java.nio` and `java.nio.file` packages.

Code Snippet

```
Path path = Paths.get("data.txt");
List<String> lines = Files.readAllLines(path);
lines.forEach(System.out::println);
```

Exercise 06 – NioRead.java
Use `Files.readAllLines()` to read a file and count the number of lines.

Files and Paths (java.nio.file)

Key Points
• `Path` represents file or directory location.
• `Files` provides utility methods: `exists()`, `copy()`, `move()`, `delete()`, `walk()`.
• Prefer NIO classes for modern file handling.

Code Snippet

```
Path source = Paths.get("input.txt");
Path target = Paths.get("backup/input.txt");
Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);
```

Exercise 07 – FileCopyNio.java
Use `Files.copy()` and `Files.move()` to back up and relocate a file.

Channels and Buffers

Key Points
• **Channels** enable faster I/O using direct buffers and memory mapping.
• **Buffers** store data temporarily before writing or after reading.
• Key classes: `FileChannel`, `ByteBuffer`, `MappedByteBuffer`.
• Must flip buffers between read and write modes.

Code Snippet

```
try (FileChannel channel = FileChannel.open(Paths.get("data.txt"))) {
    ByteBuffer buffer = ByteBuffer.allocate(64);
    channel.read(buffer);
    buffer.flip();
    System.out.println(new String(buffer.array(), 0, buffer.limit()));
}
```

Exercise 08 – ChannelBuffer.java
Read a file using `FileChannel` and `ByteBuffer` instead of streams.

Memory-Mapped Files

Key Points
• Memory-mapped files allow direct file I/O through memory, improving performance.
• Suitable for large data files or random-access operations.
• Use `FileChannel.map()` to map file regions into memory.

Code Snippet

```
try (FileChannel fc = FileChannel.open(Paths.get("data.txt"),
StandardOpenOption.READ)) {
    MappedByteBuffer mbb = fc.map(FileChannel.MapMode.READ_ONLY, 0,
fc.size());
    for (int i = 0; i < fc.size(); i++)
        System.out.print((char) mbb.get(i));
}
```

Exercise 09 – MemoryMapDemo.java
Use a memory-mapped file to read and display a large text file's content.

Certification Tips

Key Points
• Classic I/O is stream-based; NIO is buffer and channel-based.
• `Files` and `Paths` simplify modern file operations.
• `transient` skips serialization; `serialVersionUID` ensures class version compatibility.
• Always close I/O resources using try-with-resources.
• Expect questions comparing I/O and NIO, and on object serialization behavior.

Code Snippet

```
Path path = Paths.get("example.txt");
Files.write(path, List.of("Java", "I/O", "NIO"));
System.out.println(Files.readAllLines(path));
```

Exercise 10 – CertificationPractice.java
Compare performance and syntax differences between traditional `FileReader` and `Files.readAllLines()` for reading large files.

🧩 Module – Object Serialization Deep Dive (Custom Serialization, transient, Externalizable)

Serialization Recap

Key Points
• Serialization converts an object's state into a byte stream.
• Deserialization reconstructs the object from that stream.
• Classes must implement `Serializable`.
• Default serialization saves all non-static, non-transient fields.

Code Snippet

```
class Student implements Serializable {
    String name;
    int age;
}
```

Exercise 01 – BasicSerialize.java
Create a `Student` class and serialize it to a file named `student.ser`.

Role of serialVersionUID

Key Points

• `serialVersionUID` identifies the version of a class.
• If not defined, the JVM generates one dynamically (not recommended).
• Mismatched versions between serialized and current classes cause `InvalidClassException`.
• Always define it explicitly.

Code Snippet

```
class Student implements Serializable {
    private static final long serialVersionUID = 1001L;
    String name;
}
```

Exercise 02 – VersionConflict.java

Change the class structure and observe the effect of missing or mismatched `serialVersionUID`.

transient Keyword

Key Points
• Fields marked as `transient` are not serialized.
• Useful for sensitive or derived data (e.g., passwords, cached values).
• Default values are assigned during deserialization.

Code Snippet

```
class Account implements Serializable {
    String username;
    transient String password;
}
```

Exercise 03 – TransientField.java
Serialize an object containing a transient field and verify its value after deserialization.

Customizing Serialization with writeObject and readObject

Key Points
• Define `writeObject(ObjectOutputStream)` and `readObject(ObjectInputStream)` to customize serialization.
• These methods let you encrypt data, validate fields, or modify the serialization format.
• Must be declared as `private`.

Code Snippet

```
class User implements Serializable {
    private String name;
    private transient String password;
    private void writeObject(ObjectOutputStream oos) throws IOException {
        oos.defaultWriteObject();
        oos.writeObject("ENC-" + password);
    }
    private void readObject(ObjectInputStream ois) throws IOException,
ClassNotFoundException {
        ois.defaultReadObject();
        password = ((String) ois.readObject()).substring(4);
    }
}
```

Exercise 04 – CustomSerialize.java
Implement custom serialization to encrypt and decrypt a password field.

Serialization Order and Object Graphs

Key Points
• Serialization handles entire object graphs — all referenced objects are serialized recursively.
• Cyclic references are handled automatically.
• Ensure all referenced objects are serializable; otherwise, `NotSerializableException` is thrown.

Code Snippet

```
class Department implements Serializable {
    Employee emp;
}
class Employee implements Serializable {
    String name;
}
```

Exercise 05 – ObjectGraph.java
Create a `Department` object containing an `Employee` and serialize the entire hierarchy.

Preventing Serialization

Key Points
• Classes can opt out of serialization by defining `writeObject()` to throw an exception.
• Use this to prevent misuse of sensitive or non-persistent types.

Code Snippet

```
private void writeObject(ObjectOutputStream out) throws IOException {
    throw new NotSerializableException("Serialization not allowed for this
class");
}
```

Exercise 06 – BlockSerialization.java
Implement a class that explicitly blocks its serialization and test the exception.

The Externalizable Interface

Key Points

• `Externalizable` provides full control over serialization process.

• Requires implementing two methods:

– `writeExternal(ObjectOutput out)`

– `readExternal(ObjectInput in)`

• You must manage the reading and writing of all fields manually.

Code Snippet

```
class Employee implements Externalizable {
    String name;
    int id;
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeObject(name);
        out.writeInt(id);
    }
    public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
        name = (String) in.readObject();
        id = in.readInt();
    }
}
```

Exercise 07 – ExternalizableDemo.java

Implement an `Externalizable` class and manually write and read all its fields.

Difference: Serializable vs Externalizable

Key Points

Feature	Serializable	Externalizable
Control	Automatic	Manual
Performance	Moderate	Fast for custom handling
Default Behavior	Uses reflection	Developer-defined
Versioning	Uses serialVersionUID	Developer-managed
Constructors	Not invoked	No-arg constructor invoked

Code Snippet

```
Employee e = new Employee();
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("emp.ser"));
oos.writeObject(e);
```

Exercise 08 – CompareInterfaces.java
Demonstrate both interfaces and identify when `Externalizable` provides performance benefits.

Serialization Best Practices

Key Points
• Always define `serialVersionUID` explicitly.
• Avoid serializing large object graphs unnecessarily.
• Mark derived or sensitive data as `transient`.
• Implement `readResolve()` for singleton classes to preserve single instance.
• Test deserialization compatibility across versions.

Code Snippet

```
protected Object readResolve() {
    return Singleton.INSTANCE;
}
```

Exercise 09 – SingletonSerialize.java
Implement a serializable singleton and use `readResolve()` to maintain the single instance after deserialization.

Certification Tips

Key Points
• `transient` → skips serialization; default value restored on read.
• `serialVersionUID` → version control for classes.
• `Externalizable` → must implement `writeExternal()` and `readExternal()`.
• `readResolve()` → ensures singleton integrity.
• Expect exam questions comparing `Serializable` vs `Externalizable`, and on `writeObject`/`readObject` usage.

Code Snippet

```
class Config implements Serializable {
    private static final long serialVersionUID = 1L;
    private void readObject(ObjectInputStream ois) throws Exception {
        ois.defaultReadObject();
        System.out.println("Custom read logic executed");
    }
}
```

Exercise 10 – CertificationPractice.java
Explain what happens if a `transient` field is modified after serialization but before deserialization.

🧩 Module – Localization and Resource Bundles (Locale, ResourceBundle, Formatting, I18N)

Introduction to Localization (I18N)

Key Points
• **Internationalization (I18N)** means designing software that can be adapted to different languages and regions without code changes.
• **Localization (L10N)** means adapting the program for a specific locale — language, region, date, currency, and number formats.
• Java provides robust APIs for localization via `Locale`, `ResourceBundle`, and formatters.
• Used extensively in enterprise and global applications.

Code Snippet

```
Locale locale = new Locale("fr", "FR");
System.out.println(locale.getDisplayName());  // French (France)
```

Exercise 01 – LocaleIntro.java
Create and print different locales like US, France, and Japan. Display their language and country codes.

Locale Class

Key Points
• `Locale` represents a specific geographic, cultural, or linguistic region.
• Created using constructors or factory methods like `Locale.of()` or `Locale.Builder`.
• Common predefined constants: `Locale.US`, `Locale.UK`, `Locale.CANADA`.
• The default locale can be set using `Locale.setDefault()`.

Code Snippet

```
Locale india = new Locale("hi", "IN");
Locale.setDefault(india);
System.out.println(Locale.getDefault());
```

Exercise 02 – DefaultLocale.java
Change the default locale to Hindi (India) and print the language and country codes.

ResourceBundle Basics

Key Points

• `ResourceBundle` manages locale-specific resources (like text labels).
• Files are stored as `.properties` with the naming pattern:
`<baseName>_<language>_<country>.properties`.
• Java automatically picks the bundle matching the current locale.

Code Snippet

```
ResourceBundle rb = ResourceBundle.getBundle("messages", Locale.FRANCE);
System.out.println(rb.getString("greeting"));
```

Exercise 03 – BundleDemo.java

Create `messages_en_US.properties` and `messages_fr_FR.properties` with greetings, then load and print them.

ResourceBundle Lookup Rules

Key Points

• The search order for bundles follows specificity:
`<base>_language_country → <base>_language → <base>`.

• If no match found, Java falls back to the default locale or base bundle.

• Properties should be encoded in UTF-8 (Java 9+).

Code Snippet

```
ResourceBundle rb = ResourceBundle.getBundle("labels", new Locale("es",
"ES"));
System.out.println(rb.getString("welcome"));
```

Exercise 04 – LookupOrder.java

Create multiple localized bundles and print which one is loaded for various locales.

Using ListResourceBundle

Key Points
• You can also define bundles as Java classes by extending `ListResourceBundle`.
• Use when values need to be computed or loaded dynamically.

Code Snippet

```
public class Messages_en extends ListResourceBundle {
    protected Object[][] getContents() {
        return new Object[][] {
            {"welcome", "Welcome to Learnato"},
            {"bye", "Goodbye"}
        };
    }
}
```

Exercise 05 – ListResourceBundleDemo.java
Create a `ListResourceBundle` in code and load it instead of using `.properties` files.

Formatting Dates, Numbers, and Currency

Key Points
• Use `DateTimeFormatter`, `NumberFormat`, and `Currency` for locale-specific formatting.
• `NumberFormat.getCurrencyInstance(locale)` handles symbols automatically.
• Dates and numbers vary across locales (e.g., 1,000.50 vs 1.000,50).

Code Snippet

```
NumberFormat nf = NumberFormat.getCurrencyInstance(Locale.UK);
System.out.println(nf.format(12345.67));  // £12,345.67
```

Exercise 06 – CurrencyFormat.java
Display a price in at least three different currency formats (US, UK, Japan).

Localizing Dates and Times

Key Points
• Use `DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG)` with locale to adapt formats.
• Localized date/time patterns depend on country conventions.

Code Snippet

```
LocalDate date = LocalDate.now();
DateTimeFormatter fmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL)
                                         .withLocale(Locale.FRANCE);
System.out.println(date.format(fmt));
```

Exercise 07 – DateFormatLocale.java
Print today's date in FULL format for US, France, and India locales.

MessageFormat for Dynamic Text

Key Points
• `MessageFormat` allows insertion of dynamic values into localized messages.
• Placeholders `{0}`, `{1}` etc. are replaced with runtime data.

Code Snippet

```
String msg = "Hello {0}, you have {1} new messages.";
System.out.println(MessageFormat.format(msg, "Rohan", 5));
```

Exercise 08 – MessageFormatDemo.java
Create a localized message template and format it with user name and count values.

Best Practices for I18N

Key Points
• Never hardcode user-facing text; always externalize to bundles.
• Keep bundle keys consistent and meaningful.
• Avoid concatenating localized strings; use `MessageFormat`.
• Test locale fallback by temporarily changing default locales.

Code Snippet

```
Locale.setDefault(new Locale("es", "ES"));
ResourceBundle rb = ResourceBundle.getBundle("labels");
System.out.println(rb.getString("greeting"));
```

Exercise 09 – LocaleSwitch.java
Switch between locales dynamically and verify fallback behavior for missing translations.

Certification Tips

Key Points
• `Locale` = language + country.
• `ResourceBundle` automatically loads based on locale.
• `.properties` files must be in UTF-8 for Java 9+.
• `MessageFormat` supports indexed placeholders.
• Expect questions on bundle naming, lookup order, and formatting behavior.

Code Snippet

```
ResourceBundle rb = ResourceBundle.getBundle("messages", new Locale("de",
"DE"));
System.out.println(rb.getString("hello"));
```

Exercise 10 – CertificationPractice.java
Explain what happens if `messages_fr_FR.properties` is missing but `messages_fr.properties` exists for locale `fr_FR`.

🧩 Module – Understanding Basics of Java Logging API

Introduction to Java Logging

Key Points
• Java provides built-in logging through `java.util.logging` (JUL) package.
• Logging helps record application events, warnings, and errors.
• Used for debugging, auditing, and monitoring applications.
• Loggers replace `System.out.println()` with configurable, flexible logs.
Code Snippet

```
import java.util.logging.*;
public class LoggingIntro {
    private static final Logger logger =
Logger.getLogger(LoggingIntro.class.getName());
    public static void main(String[] args) {
        logger.info("Application started");
        logger.warning("Low memory warning");
    }
}
```

Exercise 01 – LoggingIntro.java
Create a simple logger and print informational and warning messages.

Logger, Handler, and Formatter

Key Points
• Logger → main entry point to record log messages.
• Handler → determines where logs are sent (console, file, etc.).
• Formatter → defines how messages are formatted.
• Common handlers: `ConsoleHandler`, `FileHandler`.
Code Snippet

```
Logger logger = Logger.getLogger("DemoLogger");
FileHandler fileHandler = new FileHandler("app.log", true);
fileHandler.setFormatter(new SimpleFormatter());
logger.addHandler(fileHandler);
logger.info("Logging to file");
```

Exercise 02 – FileLogger.java
Log messages to both console and file using appropriate handlers.

Log Levels

Key Points
• Java defines predefined log levels: `SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`.
• Control output using `logger.setLevel(Level.X)` to filter logs.
• Useful for controlling verbosity in different environments.
Code Snippet

```
logger.setLevel(Level.FINE);
logger.fine("Debug details");
logger.warning("Potential issue detected");
```

Exercise 03 – LogLevelDemo.java
Set different log levels and observe which messages appear.

Logging Configuration

Key Points
• Configuration can be done via code or properties file (`logging.properties`).
• `LogManager` reads configuration at startup.
• Useful to change log behavior without modifying code.
Sample Config (logging.properties)

```
handlers= java.util.logging.ConsoleHandler
java.util.logging.ConsoleHandler.level = FINE
.level = INFO
```

Exercise 04 – LoggingConfig.java
Load logging configuration from a file and test various levels.

Advanced Logging Practices

Key Points
• Use unique logger names per class or package.
• Avoid logging sensitive information.
• Use `logger.throwing()` or `logger.log(Level.SEVERE, msg, exception)` for errors.
• Prefer structured messages and consistent formats.
Code Snippet

```
try {
    int result = 10 / 0;
} catch (Exception e) {
    logger.log(Level.SEVERE, "Exception occurred", e);
}
```

Exercise 05 – ExceptionLogger.java
Log exceptions using `logger.log(Level.SEVERE, message, exception)`.

Certification Tips

Key Points
• `Logger`, `Handler`, `Formatter` classes are part of `java.util.logging`.
• `Level` defines severity hierarchy.
• Handlers can be chained or customized.
• Always close file handlers to avoid resource leaks.
Exercise 06 – CertificationPractice.java
Explain difference between `logger.info()` and `logger.log(Level.INFO, message)`.

Module – Using Java Annotations

What Are Annotations?

Key Points
• Annotations provide metadata about code to the compiler or runtime.
• Used for documentation, compile-time checks, or runtime processing.
• Common built-in annotations: `@Override`, `@FunctionalInterface`, `@Deprecated`, `@SuppressWarnings`, `@SafeVarargs`.
Code Snippet

```
@Override
public String toString() { return "Example"; }
```

Exercise 01 – AnnotationIntro.java
Create a class with annotated methods using `@Override`.

@Override

Key Points
• Indicates a method overrides a superclass method.
• Helps detect incorrect method signatures during compile time.
• Compiler error occurs if no matching method in superclass.
Code Snippet

```
class Parent { void greet() {} }
class Child extends Parent {
    @Override void greet() { System.out.println("Hello from Child"); }
}
```

Exercise 02 – OverrideDemo.java
Override a method from a parent class and verify compiler enforcement.

@FunctionalInterface

Key Points
• Marks an interface that contains exactly one abstract method.
• Helps validate functional interfaces used in lambda expressions.
• Optional but recommended for clarity.
Code Snippet

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Exercise 03 – FunctionalInterfaceDemo.java
Create a `@FunctionalInterface` and implement it using a lambda expression.

@Deprecated

Key Points
• Marks a method or class as obsolete or unsafe for future use.
• IDEs and compilers generate warnings when used.
• Should include documentation explaining the alternative.
Code Snippet

```
@Deprecated
void oldMethod() { System.out.println("Deprecated method"); }
```

Exercise 04 – DeprecatedExample.java
Mark an old method as deprecated and suggest a new replacement.

@SuppressWarnings

Key Points
• Tells the compiler to ignore specific warnings.
• Common parameters: `"unchecked"`, `"deprecation"`, `"rawtypes"`.
• Use sparingly to avoid masking real issues.
Code Snippet

```
@SuppressWarnings("deprecation")
public void useDeprecated() {
    oldMethod();
}
```

Exercise 05 – SuppressWarningDemo.java
Call a deprecated method with and without suppression to see warning difference.

@SafeVarargs

Key Points
• Ensures no heap pollution occurs when using varargs with generics.
• Applicable to constructors or static/final methods only.
• Added in Java 7 to enhance generic safety.
Code Snippet

```
@SafeVarargs
public static <T> void printAll(T... elements) {
    for (T e : elements) System.out.println(e);
}
```

Exercise 06 – SafeVarargsDemo.java
Create a generic varargs method and annotate it with `@SafeVarargs`.

Custom Annotations (Optional Extension)

Key Points
• You can define custom annotations using `@interface`.
• Can include default values and retention policies.
• Used in frameworks like Spring, Hibernate, etc.
Code Snippet

```
@interface Author {
    String name();
    String date() default "N/A";
}
```

Exercise 07 – CustomAnnotation.java
Define and use a custom annotation to tag author and date information.

Certification Tips

Key Points

- `@Override` ensures correct method overriding.
- `@FunctionalInterface` validates lambda compatibility.
- `@Deprecated` + `@SuppressWarnings` often appear together in questions.
- `@SafeVarargs` applicable only to final, static, or constructors.

Exercise 08 – CertificationPractice.java

Identify which annotations apply compile-time vs runtime and which allow multiple targets.